Pulsefy Dashboards — Code Review Handover

Covers the three internal-facing dashboards: **Event Organizer**, **Admin**, and **Business**. Written for a code reviewer who has not seen the codebase before. Line numbers are accurate as of this branch (`claude/fix-event-organizer-feeds-ZCzRM` , merged with `main` @ `e2e1bb6`).

For the consumer-facing app (`apps/landing-page/index.html`) and its fixes log, see `CLAUDE.md` section 7. This document does not repeat that.

---

1. Event Organizer Dashboard

File: `apps/organizer/index.html` (3,070 lines, single-file SPA, same pattern as the main app — vanilla JS, Supabase client, inline `<style>` / `<script>`)

Who uses it: Users with `profiles.role = 'organizer'` — independent event promoters who don't run a physical venue/business.

Auth flow

- On load, checks for a Supabase session (`p_token` in localStorage, same key used by the main app and business dashboard).
- No session → redirect to login.
- Session present → fetches `profiles` row, expects `role = 'organizer'` .

Features

Tab	What it does	Storage
Post creation	Up to 5 photos, 1000-char caption, optional event tag	`posts` table
Event creation	Name, date/time, venue, ticket tiers, genre, GPS, map-pin approval request	`events` table (direct Supabase insert, not via API)
Feed	View posts from followed organizers, like/comment	`posts`, related comment table
Squad Deals	Group-discount promos with templates (VIP Bundle, Group Entry, etc.)	`squad_promos` (admin-approved before going live)
Promotions	Boost an event/post into the feed/Featured-This-Weekend slot, 7/14/30-day duration	`promotions` (admin-approved)
Pricelist	Sell items/services directly (packages, catering, tickets, etc.)	`organizer_menu_items`
QR scanning	Scan attendee tickets at the door using `jsQR`	reads `bookings`, writes `checked_in`/`checked_in_at`
Settings	Verification application, Premium upgrade (R220/mo), payout bank details, social links	`profiles`, `verifications`

Things worth flagging in review

- **Event creation writes directly to Supabase from the client** (no API layer), unlike events read paths which go through `api/events/index.js`. This is consistent with the rest of the codebase (CLAUDE.md roadmap item "Frontend API service layer" is still open) but worth knowing — RLS is the only thing preventing abuse here.
- **Free-tier post/menu-item limits are enforced client-side only** (a usage counter, not a hard block). Confirm whether the relevant `/api/*` insert paths enforce limits server-side, or whether a user can bypass via direct Supabase calls.
- **QR ticket format is simple and guessable**: `PULSIFY:{booking_ref}:{event_id}:VALID` (see `api/payments/index.js` — same format used by both organizer and business scanners). No HMAC/signature. Low risk currently since check-in requires the organizer to own the event, but worth a second look before ticket volume scales.

---

2. Admin Dashboard

Two separate files — make sure the reviewer knows which is which:

2a. `apps/admin/index.html` (3,010 lines) — the real admin console

Auth: Supabase email/password or Google sign-in, then a hard role check (`apps/admin/index.html:1024` — `profiles.role` must be `'admin'`, otherwise signed out immediately).

Tabs:

Tab	Purpose	Backing table(s)
Users	Search/filter all users, role changes, suspend, grant trial months	`profiles`
Business coordinates	Bulk-fix bad/missing lat-lon, manual geocode, add new business pins	`businesses`
Event approval	Approve/reject pending events, manually add events with auto-geocode	`events`
Leads	Scraped-lead pipeline (new → contacted → converted/ignored), manually attach events to a lead	`scraped_leads`, `lead_events` (via `api/admin/index.js`)
Verifications	Approve/reject ✅ badge applications, triggers email via `api/email.js`	`verifications`
Banners	Create/edit promotional banners shown in-app	`banners`
Notifications	Compose + send push notifications, template presets, audience targeting	`notifications`
Reports	Review user-submitted reports on events/businesses/posts	`reports`

Promotions / Squad Deals / Profile Claims / Location Requests	Approval queues feeding the organizer/business submission flows above	respective tables
System	Quicket event seeding, OSM lead scraping, delete mock events, social link config	various

2b. `apps/admin/admin-panel.html` (443 lines) — admin account creation only

A much smaller, separate page just for creating new admin accounts (`/api/admin/create-admin`) and listing existing admins. Confirm with the team whether this is still needed or whether `apps/admin/index.html` 's Users tab should absorb it — having two separate admin entry points is a bit confusing for a reviewer (and for onboarding new admins).

Things worth flagging in review

- **Bulk geocode/coordinate fix has no rollback** — if it partially fails midway through a batch, some businesses are left fixed and others aren't, with no record of which.
- **Notification compose has no audience-size preview** before sending — easy to accidentally broadcast to "All users" when "By city" was intended.
- **Leads pagination** defaults to 20/page with no upper bound check on the requested page — fine today, worth a guard before the leads table grows.

---

3. Business Dashboard

Files:

- `apps/business/login.html` (335 lines) — auth + registration
- `apps/business/index.html` (3,269 lines) — main dashboard
- `apps/business/menu.html` (453 lines) — public-facing ordering page customers see
- `apps/business/order-confirmation.html` (215 lines) — post-checkout page

Who uses it: Venue/restaurant owners (`profiles.role = 'business'`).

Auth flow — known fragile point

`apps/business/login.html:157,174,259` all call `/api/auth/ensure-business-profile` after Supabase auth succeeds, to make sure a `businesses` row exists and is linked to the new `profiles` row. **If this call fails silently, the user lands on the dashboard with no business profile and the UI has nothing to show.** This is the single highest-value thing to verify works end-to-end in review — register a fresh test business account and confirm the dashboard populates correctly on first login.

Features

Tab	What it does	Storage
Home	Stats (views/saves/followers), quick actions, pending-orders preview, copyable public menu link	computed from below
Posts	Same pattern as organizer posts	`posts`

Events	Businesses can also host events (same `events` table as organizer flow)	`events`
Menu	Item CRUD (name, price, photos, category, availability toggle), pickup hours editor	**`localStorage` (`{bizId}_menu`, `{bizId}_hours`)**
Orders	Incoming orders from the public menu page, status workflow (pending → ready → completed/cancelled), QR scan to confirm	**`localStorage` (`{bizId}_orders`)**
Promotions / Squad Deals	Same submit-for-approval pattern as organizer	`promotions`, `squad_promos`
Analytics	Premium-gated, last-7-days view	(not verified — confirm data source with the team)
Settings	Profile edit, verification application, Premium (R360/mo), payout bank account, location pin (admin-approved), social links	`profiles`, `businesses`, `verifications`

`order-confirmation.html` — explicitly demo mode

Line ~169 says "No real charge — demo mode" directly in the UI copy. **Confirm with the team whether Paystack is meant to be live yet** — per `CLAUDE.md` roadmap section C, server-side payment verification is flagged as the **highest architecture priority**, currently disabled, bookings auto-confirm without payment. This affects both the ticket-purchase flow (`api/payments/index.js`) and the business order flow.

Things worth flagging in review

- **Menu, pickup hours, and orders all live in `localStorage`, not Supabase.** This means: clearing browser data or switching devices loses all menu/order history, and there's no way for the admin dashboard or any backend job to see business order volume. This is the biggest structural risk in this dashboard — confirm whether this is intentional (e.g. a prototype phase) or should be migrated to a real table before more businesses onboard.
- Free-tier menu item limit (10 items) is the same client-side-only pattern as the organizer post limit — same caveat applies.
- "Use My Location" for the business pin has no visible error state if the browser denies geolocation permission.

---

4. Backend API surface used by these dashboards

File	Lines	Endpoints
`api/shared.js`	109	Shared Supabase clients (`sb()` service-key, `sbAs(token)` RLS-scoped), `authUser()`, `rateLimit()`, CORS, Sentry hook
`api/admin/index.js`	796	Lead ingest/list/update, lead→event creation, admin-only auth gate
`api/events/index.js`	164	Events list/search/detail, businesses list — public read paths

`api/payments/index.js`	311	Ticket purchase, booking lookup, ticket validation/check-in, Paystack webhook
`api/squads/index.js`	339	Squad CRUD, leaderboard, check-in points, invites, plans
`api/email.js`	258	Nodemailer templates: welcome, verification approved/rejected

`api/squads.js` (5 lines, repo root `api/`) is an orphaned shim — not referenced by `vercel.json`'s routes. Flagged to the project owner as a removal candidate; **not removed yet pending confirmation**, leaving it as-is for this handover.

Confirmed code-level issue worth a reviewer's attention

`api/events/index.js:50`:

```
if (search) query = query.textSearch('id', search, { type: 'websearch', config: 'english' });
```

Full-text search runs against the `id` column instead of `name` / `description`. Event search by keyword likely returns wrong/empty results — worth verifying behavior in review and fixing the column reference.

CORS

`api/shared.js` sets `Access-Control-Allow-Origin: *` for all API responses. Fine for a single-domain PWA but worth a deliberate decision (vs. an oversight) before this goes further into production hardening.

---

5. Cross-cutting notes for the reviewer

- All three dashboards share the same auth/session pattern: Supabase JWT in `localStorage` (`p_token`), role read off `profiles.role`, hard role check on dashboard load (redirect/sign-out if mismatched).
- All three follow the same **submit → admin-approval queue** pattern for anything visible to end users (events, promotions, squad deals, verifications, location pins). The admin dashboard is the single approval surface for all of them.
- Rate limiting (`api/shared.js`) is in-memory per serverless instance, not distributed — acceptable today, won't hold under multi-region scale.
- See `CLAUDE.md` section 8 (Roadmap) for the team's own prioritized list of known gaps — notably **server-side payment verification** (architecture priority #1) and **frontend API service layer** (#2), both of which intersect directly with what's documented above.